ParakhMitra — The Build Journey

How this project was actually built, in order, from the starting point to a live deployment — with the real commands, configuration, decisions, and every problem we hit and fixed.

This is the companion to `CODEBOOK.md`. The Codebook explains **how the finished system works**; this document is the **construction journal** — the sequence of steps we followed, so a teammate could understand (or reproduce) the whole build. It's deliberately detailed: exact commands, SQL, dashboard settings, and the debugging log.

A note on method: we built in **strict priority order** — each layer had to work end-to-end before starting the next. That discipline is why the project never became a pile of half-finished features.

Contents

Stage 0 — The starting point

We did **not** start from a blank folder. When the build began, the repository already contained:

- A working **backend** (`backend/`, FastAPI in Python): a single endpoint `POST /api/scans` that accepted `{ front_path, back_path }`, fetched those two images from Supabase Storage, called Google **Gemini** to read the label, ran a **placeholder** rule check, saved a row to a Supabase `scans` table, and returned the result.

- A **frontend** (`frontend/`) that was still the **default Vite + React starter template** — the spinning-logo demo page. None of the real app existed yet.
- A **design system** in `DESIGN.md` (a navy "institutional" palette, the Inter font, compliance green/amber/red, spacing and component specs) that we had to implement faithfully.
- The product brief: a **mobile** app called **ParakhMitra** for Legal Metrology inspection officers, with a hard rule — **inspection records must be immutable** (no one can ever edit or delete a scan's data or verdict) — and a "no fake data" rule (every number on screen must come from real Supabase/backend data).

So Stage 0's real work was **reading and understanding** what existed: `DESIGN.md`, the backend's `scans.py`, `gemini_service.py`, `supabase_service.py`, `config.py`, the rule `engine.py`, and the frontend's `package.json`/`App.tsx`. You cannot safely extend a system you haven't read.

Stage 1 — Data model + the Supabase database

Goal: give every scan an owner, add a people/roles table, and lock everything down so records are immutable.

1a. Minimal backend change — attach an owner to each scan

We threaded a `user_id` through the existing pipeline **without touching** the Gemini extraction or the rule logic:

- `backend/app/schemas/scan.py` — added an optional `user_id` to the `ScanRequest`.
- `backend/app/api/scans.py` — read it from the request and passed it along.
- `backend/app/services/supabase_service.py` — stored it on the row (with a fallback for older DB schemas).

This was deliberately the *smallest possible* change — the brief said not to disturb the working extraction/rule pipeline.

1b. The database schema (run once in Supabase)

We wrote all the database setup into one idempotent SQL file, `supabase/schema.sql`, and ran it in the **Supabase Dashboard -> SQL Editor**. It does five things:

1. `alter table public.scans add column ... user_id` — the owner column.
2. Creates the `profiles` table (one row per user: `id`, `email`, `full_name`, `role` ∈ {admin, officer, none}, `status` ∈ {active, inactive}, `created_at`).
3. A **trigger** (`handle_new_user`) that runs automatically on first Google sign-in and derives the role from the email: an **admin allow-list** -> `admin`; anything ending in `@ves.ac.in` -> `officer`; everyone else -> `none` (no access).
4. **Row Level Security** turned on: clients may only **read** scans (their own, or all if admin) — there is deliberately **no** insert/update/delete policy for clients, which is what makes records immutable. Profiles are readable by self/admin and editable only by admins.

5. Creates the `evidence-photos` storage bucket and a policy letting logged-in users upload to it.

A subtle-but-important helper in that file: `is_admin()` is declared `security definer` so it can read `profiles` without triggering the profiles RLS policy that *calls* `is_admin()` — otherwise it would recurse forever. (This exact trap bites many Supabase projects.)

Design decisions made here: - *Roles live in the database, not the code*, so an admin can change them later without a redeploy. - *Immutability is enforced by the database itself* (RLS), not by hoping the app behaves — the strongest possible guarantee. - *The backend writes with a master "service-role" key* that bypasses RLS; the frontend uses a limited "anon" key that RLS always governs.

Stage 2 — Frontend foundation

Goal: replace the starter template with the real app's skeleton — tooling, a login/identity system, the design system, and navigation.

2a. Dependencies

```
cd frontend
npm install @supabase/supabase-js react-router-dom
```

`@supabase/supabase-js` talks to the database/auth/storage from the browser; `react-router-dom` provides screen routing.

2b. The building blocks we created

- `src/lib/supabase.ts` — one shared Supabase client, configured to persist and auto-refresh the login session.
- `src/lib/types.ts` — the TypeScript shapes for scans, profiles, extracted data, violations.
- `src/lib/api.ts` — helpers to upload a photo to storage and to call the backend.
- `src/auth/AuthContext.tsx` — the app-wide "who is logged in and what's their role" system, listening to Supabase's auth state and loading the matching profile.
- `src/index.css` — the **design system** translated from `DESIGN.md` into CSS variables and component classes (navy palette, Inter font, cards, pills, buttons, the mobile "app shell" and bottom nav).
- `src/components/` — reusable UI (icons, avatar, status pills, empty states, the `Layout` frame with bottom navigation).
- `src/App.tsx` + `src/main.tsx` — routing wired up, wrapped in the router and the auth provider, with a gate: show a loader while checking auth, else Login, else the app, else an "access denied" screen.

We verified constantly with `npx tsc --noEmit` (type-check), `npx eslint src` (lint), and `npm run build` — keeping the tree green at every step rather than debugging a mountain at the end.

Stage 3 — Priority 1: the core scan loop

Goal (the brief's #1): login -> new scan -> real result, working end-to-end, before anything else.

- **Login** (`screens/Login.tsx`): a single "Continue with Google" button (`supabase.auth.signInWithOAuth`).
- **New Scan** (`screens/NewScan.tsx`): two photo tiles (front/back, opening the rear camera on mobile), a "Product Category" dropdown, and a "Scan for Compliance" button. On submit it uploads both photos to the `evidence-photos` bucket as `${uuid}.jpg`, then POSTs the two filenames (+ the user id) to `/api/scans`, with a loading state.
- **Results** (`screens/Results.tsx`): the real backend response — the compliance status shown prominently, the extracted fields labelled, and the violations as cards showing the exact `field`/`issue`/`rule_ref`. Read-only.

At this point we needed the **external setup** to actually test a login: the Supabase Google provider and Google Cloud OAuth (documented in Stage 12/13 and `SETUP.md`), plus the env files (Appendix B).

Decision: photos upload *directly from the phone to storage*; only the tiny filenames go to the backend. This keeps the backend fast and cheap (it never carries megabytes of image).

Stage 4 — Priority 2: oversight screens

Goal (the brief's #2): the read-and-review screens, each scoped by role.

- **Dashboard** (`screens/Dashboard.tsx`): a welcome with the real Google name/photo, stat cards (total / compliant / flagged), and recent scans — **scoped**: an officer sees only their own scans, an admin sees all (labelled "system-wide"). Empty states when there's no data.
- **History / Inspections** (`screens/History.tsx`): a searchable list with All / Compliant / Violations filter tabs, same role scoping, tapping a row opens the detail.
- **Scan Detail** (`screens/ScanDetail.tsx`): a read-only view of one record.
- **Profile** (`screens/Profile.tsx`): the logged-in user's real identity and a working sign-out.

The role scoping lives in a small hook, `hooks/useScans.ts`: for non-admins it adds `.eq('user_id', me)` to the query; admins skip that filter. (The database's RLS enforces the same scoping independently, so it can't be bypassed from the browser.)

Stage 5 — Priority 3: admin user management

Goal (the brief's #3, admin-only): a screen for admins to manage people — never scans.

- `screens/Users.tsx`: lists all `profiles`; an admin can edit a user's `full_name` and `role` (officer/admin) and toggle `status` (active/inactive — inactive users are denied at login). Changes write to `profiles`, with a confirmation step and error handling.
- It is **hidden from officers** in the bottom navigation *and* protected at the route level (an officer who types the URL is redirected home).
- Admins **cannot touch any scan record** — there is deliberately no such capability anywhere.

Stage 6 — The real rule engine

Goal: replace the placeholder checks with the real **8 Legal Metrology (Packaged Commodities) Rules, 2011**, without changing the AI or anything else.

We rewrote `backend/app/rules/engine.py` so the rules are a **plain, editable list** — each entry has the `field`, the exact `rule_ref` (e.g. `Rule 6(1)(a)`), a plain-English `issue`, and a tiny `check` function. The eight cover manufacturer, product name, net quantity + unit, mfg/pack date, MRP, MRP-inclusive-of-tax, consumer care, and forbidden units (dozen/score/gross). We added editable unit vocabularies (standard units, common-spelling aliases, prohibited units) and **guards so two related rules don't "double-fire."**

We tested the engine directly with sample data (empty label -> 6 violations; a `dozen` unit -> the two expected violations; `GM` normalised to `g`; etc.) before wiring it live.

One gap we found by testing: Rule 6(1)(b) needs a `product_name`, but the AI schema didn't yet return one — so *every* scan flagged. With the user's approval we made the smallest possible extraction change: added `product_name` to the `ExtractedData` schema and one line to the Gemini prompt. Re-testing a real, well-declared label then returned **compliant**. This proved the whole extract->judge pipeline end-to-end.

Decision (important for the presentation): the AI only *reads* the label; a deterministic Python engine *judges* it, because a legal verdict must be identical every time and point at an exact rule. We also explicitly **excluded FSSAI** rules — that's a different regulator (food safety), not Legal Metrology.

Stage 7 — Product categories

Goal: let officers tag a scan with a real product category, stored and shown, without changing the rule logic.

- Frontend: the dropdown now lists seven real categories (General, Food & Beverages, Personal Care & Cosmetics, Household & Cleaning, Electronics & Appliances, Textiles & Garments, Other); the choice is sent with the scan and shown in History and the detail view.
- Backend: `category` threaded through the request -> saved on the row. The engine gained an **empty** `CATEGORY_RULES` hook (a place for future per-category rules) but **every category runs the same 8 rules today**.
- Database: `alter table public.scans add column if not exists category text;`

This stage is also where we hit the "column does not exist" errors and learned the discipline of running the migration *before* deploying code that reads the new column (Appendix A).

Stage 8 — Switching the AI model

While testing, Gemini occasionally returned transient `503 UNAVAILABLE` ("model overloaded"). We switched to **`gemini-3.5-flash-lite`** (higher rate limits, lower latency, plenty powerful for reading a label), and made the model name an **environment variable** (`GEMINI_MODEL`) so it can be changed or rolled back without editing code. This was a one-line change in `backend/app/config.py`.

Stage 9 — The PDF Improvement Notice

Goal: generate a formal, printable "Legal Metrology Improvement Notice" PDF for any scan.

9a. Choosing the engine

The obvious library, **WeasyPrint**, needs heavy system libraries that are painful on Windows (our dev machine) and on free hosting. We chose **`xhtml2pdf`** instead — pure Python, installs with just `pip`, works everywhere — and kept the "design an HTML template, fill it, convert to PDF" approach.

```
cd backend
.\venv\Scripts\python.exe -m pip install xhtml2pdf jinja2
```

9b. What we built

- `backend/app/services/report_service.py` — a Jinja2 HTML template (navy header, a green/red verdict banner, a violations table with rule references, the two evidence photos, a formal notice paragraph, signature block) plus `generate_notice_pdf(...)`.
- A read-only `GET /api/scans/{scan_id}/notice` endpoint returning the PDF as a download.
- A **"Download Improvement Notice"** button on the scan detail, for both officers and admins.

9c. The debugging saga (this is the honest part)

The first versions looked wrong, and fixing them taught us the tool's quirks:

1. **Photos overflowed the page** — we'd only set an image *width*, so tall phone photos ballooned vertically. Fix: normalise each image with **Pillow** (downscale, re-encode) and set an explicit **width *and* height** computed to fit a fixed box.
2. **The whole layout scrambled** (photos floating over the header, sections overlapping) — caused by custom `@frame` CSS that xhtml2pdf mishandles. Fix: remove the custom frames and let content flow with a simple `@page`.
3. **The "Evidence" heading orphaned** onto the next page from its photos — fixed with `page-break-inside: avoid` on that block.

We verified each fix by **rendering the PDF pages to images** and actually looking at them, not by guessing.

Stage 10 — Evidence photos + strict admin/officer separation

Two related polish tasks:

- **Show the evidence images in the scan detail** (for both roles): the `evidence-photos` bucket is public, so we build public URLs from the stored filename and display the front/back photos with graceful placeholders if one fails to load.

- **Strict role separation:** admins are supervisors who **do not scan** — we removed the New Scan tab/button for admins *and* redirected the `/scan` and `/results` routes to the dashboard so an admin can't reach the scan screen even by typing the URL. Officers are unaffected.

Stage 11 — Security hardening (real authentication)

Before going public, we did a security review and found the big one: **the backend endpoints had no authentication.** On a laptop that's fine; on the open internet it means anyone could call them. We fixed it:

- **JWT auth on both endpoints:** a `get_current_user` dependency validates the Supabase login token (`Authorization: Bearer …`) and confirms an *active* officer/admin profile — else 401/403. The frontend now attaches the token on every backend call.
- **Anti-spoofing:** the scan owner is taken from the *verified token*, never from the request body.
- **Owner-or-admin check** on the notice endpoint (mirroring RLS, which the service-role backend bypasses).
- **Rate limiting:** a per-user cap (20 scans/minute) on the Gemini-billed endpoint.
- **Error sanitisation:** generic messages to clients; detailed errors only in server logs.

We verified with an in-process test client that missing/bad tokens now return **401**. (One item was left as a deliberate, documented choice: the evidence bucket stays public — low-sensitivity product photos with random filenames.)

Stage 12 — Deployment

Goal: put both programs on the internet. Frontend -> **Vercel**, backend -> **Render**, database already on **Supabase**.

12a. Prepare the code

- Made CORS configurable: `backend/app/config.py` reads `CORS_ORIGINS` from the environment (falls back to localhost).
- Added `render.yaml` (the backend's Render blueprint) and `frontend/vercel.json` (an SPA rewrite so deep-link refreshes work).
- Wrote `DEPLOY.md`.
- Confirmed secrets are gitignored (`.env`, `.env.local` are **not** in the repo), then committed and pushed everything to GitHub (`pxrthj/sih-d9a`).

12b. Backend on Render

New Web Service from the GitHub repo, with these settings (the two people usually miss are **Root Directory** and the **Start Command**):

- **Root Directory:** `backend`
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- **Instance Type:** Free · **Region:** Singapore
- **Environment variables:** `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `GEMINI_API_KEY`, `GEMINI_MODEL=gemini-3.5-flash-lite`, `CORS_ORIGINS` (placeholder for now), `PYTHON_VERSION=3.12.7`

It went live at **`https://sih-d9a.onrender.com`**; we confirmed with `…/health`.

12c. Frontend on Vercel

Import the same repo, then:

- **Root Directory:** `frontend` (Vite auto-detected)
- **Environment variables:** `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, and `VITE_API_BASE_URL=https://sih-d9a.onrender.com`

It went live at **`https://parakh-mitra.vercel.app`**.

12d. Wire them together

- **Render -> `CORS_ORIGINS`** set to the Vercel URL (so the browser is allowed to call the backend).
- **Supabase -> Authentication -> URL Configuration: Site URL** and **Redirect URLs** set to the Vercel URL (so Google login returns to the live site, not localhost).
- Google OAuth needed **no** change (its redirect URI stays the Supabase callback).

The order matters: deploy the backend first (to get its URL for the frontend), then the frontend (to get its URL for CORS + Supabase redirect).

Stage 13 — Publishing Google login for the whole team

The Google OAuth app started in **Testing** mode (only listed test users can sign in). To let teammates try it, we **published the OAuth consent screen to Production** (Google Cloud -> Google Auth Platform -> Audience -> Publish). For a login-only app using basic email/profile scopes, this needs no Google verification review.

Important nuance we documented: publishing OAuth lets *anyone sign in with Google*, but the app's **own** access rule still applies — only the admin allow-list and `@ves.ac.in` emails get in; everyone else is signed in but denied until an admin promotes them from the Users screen.

Appendix A — The troubleshooting log

Every real problem we hit, and the fix. This is the most useful page for a teammate.

Symptom	Cause	Fix
`index.html` shows a blank page when opened	Opened the file directly instead of via the dev server	Run `npm run dev` and open `http://localhost:5173` — a Vite app only works when *served*
`uvicorn : The term 'uvicorn' is not recognized`	Using the system Python, which doesn't have the backend's libraries	Use the venv: `.\venv\Scripts\python.exe -m uvicorn app.main:app …`
History/Dashboard: *"column scans.front_path does not exist"*	The frontend queried columns the table didn't have	Select only real columns; the table uses one combined `storage_path`
New Scan / History error: *"column scans.category does not exist"* (`PGRST204`)	Code shipped before the DB migration	Run `alter table public.scans add column if not exists category text;` first
Wipe SQL failed: *"Direct deletion from storage tables is not allowed"*	Supabase blocks raw deletes on `storage.objects`	Delete files via the Storage API/dashboard; use SQL only for `scans`/`auth.users`
Scan error: *Gemini `503 UNAVAILABLE`*	Transient model overload	Retry; and we switched to `gemini-3.5-flash-lite` for higher limits
PDF: evidence photos "all over"	Image had only a width, so portrait photos overflowed	Pillow-normalise + set explicit width **and** height (bounded box)
PDF: sections overlapping / photos over the header	Custom `@frame` CSS that xhtml2pdf mishandles	Remove custom frames; use a simple `@page` and let content flow
After Google sign-in: *"localhost refused to connect"*	Supabase **Site URL** still pointed at localhost	Set Site URL + Redirect URLs to the live Vercel URL
Render build: *"Could not open requirements file"*	**Root Directory** not set to `backend`	Set Root Directory to `backend`, redeploy
On phone: *"failed to fetch"*	(a) `CORS_ORIGINS` didn't match the Vercel origin; (b) Render free-tier cold start	Set `CORS_ORIGINS` to the exact Vercel URL; warm the backend via `/health`
`GET /favicon.ico 404`, `HEAD / 405` in logs	Browser favicon probe / health probe	Harmless — not errors

The recurring lesson: most "bugs" in deployment were **configuration**, not code — a URL that didn't match, a directory not set, a migration not run. Change one setting, redeploy, retest.

Appendix B — Complete setup & run reference

Environment variables

`frontend/.env.local`:

```
VITE_SUPABASE_URL=<your supabase project url>
VITE_SUPABASE_ANON_KEY=<your supabase anon/public key>
VITE_API_BASE_URL=http://127.0.0.1:8000 # locally; the Render URL in production
```

`backend/.env`:

```
SUPABASE_URL=<your supabase project url>
SUPABASE_SERVICE_ROLE_KEY=<service-role key — server only, never in the frontend>
GEMINI_API_KEY=<your gemini key>
GEMINI_MODEL=gemini-3.5-flash-lite # optional; this is the default
CORS_ORIGINS=http://localhost:5173 # locally; the Vercel URL in production
```

One-time external setup

1. **Database:** run `supabase/schema.sql` in the Supabase SQL Editor (edit the admin allow-list first). Ensure the `category` column exists.
2. **Google OAuth:** create a Web OAuth client in Google Cloud (redirect URI = the Supabase auth callback), add test users (or publish to Production), enable the Google provider in Supabase with the client id/secret, and set Supabase Site URL + Redirect URLs.
3. **Env files:** fill in the two files above.

Run locally (two terminals)

Backend (from `backend/`):

```
.\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Frontend (from `frontend/`):

```
npm run dev
```

Open **http://localhost:5173**. Backend health: **http://127.0.0.1:8000/health**. API docs: **http://127.0.0.1:8000/docs**.

Reset all data for a clean test

In the Supabase SQL Editor:

```
delete from public.scans;
delete from auth.users; -- also clears profiles via cascade; you'll be signed out
```

Then delete the files in the `evidence-photos` bucket from the Storage UI.

The journey in one paragraph

We started from a working extraction backend and an empty frontend template. We **modelled the data and locked it down** (owner column, profiles, roles, immutable-by-RLS), **built the frontend skeleton** (auth, design system, routing), then delivered the app in strict priority order: **the core scan loop**, then **oversight screens**, then **admin user management**. We replaced the placeholder checks with the **real 8 Legal Metrology rules**, added **product categories**, and moved the AI to **Flash-Lite**. We built the **PDF notice** (fighting xhtml2pdf's quirks with real image and layout fixes), showed **evidence photos**, and enforced **strict admin/officer separation**. Before going public we **hardened security with real authentication**. Finally we **deployed** — backend to Render, frontend to Vercel, wired together by URLs and CORS — and **published Google login** for the team. Most deployment trouble was configuration, not code; the fix was always: correct one setting, redeploy, retest.